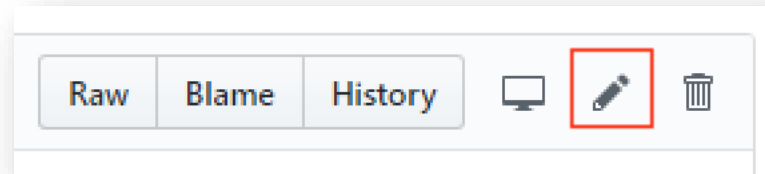


Exercise 9: Pull

For this exercise, you will practice pulling, merging, and handling conflicts. You will simulate a collaboration editing your file directly in GitHub.

Pull a Change from GitHub

1. Open GitHub. Click on your **docs.md** file to see the content.
2. Click on the Edit pen on the right.



3. Change the second line. For example, it could say, “Note that this toothbrush is totally safe to use for children, adults, pets, and wildlife.”
4. Scroll to the bottom where it says **Commit Changes**. In the top box, put a short comment, like “Added wildlife.”
5. Leave as **Commit directly to the master branch**.
6. Click **Commit Change**. Now the GitHub version has a change in its second paragraph.
7. Make sure you are in the master branch, and type `git pull`. This will get you the changes.
8. Open your editor and verify that you have the “wildlife” change.

Merge from GitHub

Now you will make changes on both GitHub and locally and make Git merge the two versions.

1. Repeat the steps in GitHub and make another change and commit it. For example, you could make the second sentence say, “Note that this toothbrush is totally safe to use for children, adults, pets, wildlife, and trees.” (I know, we are getting more absurd.)
2. Now open your editor and make a change to the first line on your local version of the file. Let’s say that marketing has asked you to change “Pavel robotic toothbrush” to “Pavel automated toothbrush”. Your first sentence can now read, “This documentation describes how to use the Pavel automated toothbrush.”
3. Now the version on GitHub has a change in the second sentence and your local copy has a change in the first sentence. Let’s say you want to get the change from GitHub. Open your terminal console in the `example` directory. Type `git pull`. Notice the error – you can’t do a pull unless you have your changes committed.
4. Add your change and then commit it. Now try `git pull` again.
5. At this point, a merge will occur, and this will require another commit. Git handles it automatically, bringing up an editor where you can make changes to merge message. Typically, you can leave the message as it is. If you have set up Git with an editor you use (such as Notepad++), then it will bring up that editor. Simply save changes and close the editor. If you

have not set up an editor, it might bring up an ancient text editor called Vim within the console itself. Fortunately, you don't need to know how to use Vim. If you see an editor in the console, just type these characters:

```
:wq
```

This means “save and quit”.

6. Now go back to the editor. Notice that it merged the changes: you should see both your local change to the first line and the GitHub change to the second line.

Merge from GitHub

Now you will make changes on both GitHub and locally and make Git merge the two versions, but this time you will modify the same line, causing a conflict, which you will have to resolve.

1. Repeat the steps in GitHub and make another change and commit it, changing the first line in both versions. For example, perhaps marketing has changed the name from “Pavel” to “Karel”.
2. Repeat the steps in your editor to change the file. Perhaps change “automated” to “automatic”. Add and commit the change.
3. Do a `git pull`. Look for the conflict message saying that the automatic merge failed.
4. Open your editor. Notice how text has been added that shows both your local changes and those from GitHub.
5. Edit the file, removing the characters that Git has added in (lines that start with <<<<, =====, or >>>>). In the end, you should have a line that uses both “Karel” and “automatic”.
6. Try a `git push` to put it up on GitHub. Notice how it gives you an error. This is because the merge has not been committed.
7. Do a `git add`, followed by a `git commit -m "Merged changes"`
8. Type `git push` again, and then verify that the change is up on GitHub.

You have successfully merged a conflict!